

CM-APP Code Review Report

GitHub: ttg92195-cmyk/CM-APP | Branch: main | Flutter Movie App (KMM)

Metric	Value
Project	CM Movies (KMM) - Flutter Movie/Series App
Framework	Flutter 3.x + Dart
Backend	Firebase (Auth + Firestore)
Total Dart Files	30+
CRITICAL Issues	5
HIGH Issues	7
MEDIUM Issues	10
LOW Issues	10
Review Date	2026-05-10

1. CRITICAL - Security Issues

These issues pose immediate security risks and should be addressed before any production deployment. Exposed credentials, hardcoded admin emails, and missing security rules can lead to unauthorized access, data breaches, and abuse of your Firebase backend.

Severity	Issue	File	Description
----------	-------	------	-------------

CRITICAL	GitHub Token Exposed	Chat/N/A	The GitHub Personal Access Token (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) was shared in plaintext. This token must be revoked immediately via GitHub Settings > Developer Settings > Personal Access Tokens. Any leaked token can be used to access private repositories, modify code, or create malicious commits.
CRITICAL	Firestore API Key Exposed	firebase_options.dart, google-services.json	Firestore API key (AIzaSyCYTCj8UBoaEj-LH_R3NjrW5S5GJC87oMo) and project configuration are committed to the repository. While Firestore API keys are designed to be somewhat public, Firestore Security Rules must be properly configured to prevent unauthorized read/write access. If rules are set to allow all, anyone can read or delete your entire database.
CRITICAL	Admin Email Hardcoded	app_config.dart:36	Admin email "guyg20985@gmail.com" is hardcoded in _adminEmailMap. This is a severe security vulnerability because anyone reading the source code (which is public on GitHub) can identify the admin account and attempt targeted attacks. Use Firestore Custom Claims or a Firestore "admins" collection instead.
CRITICAL	No .gitignore File	Project Root	The repository has no .gitignore file. Sensitive files like google-services.json, firebase_options.dart, and build artifacts are committed. A proper Flutter .gitignore should exclude android/app/google-services.json, build/, .dart_tool/, and environment-specific config files.
CRITICAL	No Firestore Security Rules	N/A (Missing)	No firestore.rules file exists in the repository. Without explicit rules, Firestore defaults to deny-all (if in production mode) or allow-all (if in test mode). If test mode, any client can read/write/delete all data including user profiles, bookmarks, and admin content.

2. HIGH - Architecture and Design Issues

These issues affect the maintainability, scalability, and reliability of the application. They represent fundamental design problems that will become increasingly difficult to fix as the codebase grows. Addressing them early will save significant refactoring effort later.

Severity	Issue	File	Description
HIGH	Service Instantiation Anti-Pattern	Multiple Screens	Each screen creates new service instances (FirestoreContentService(), BookmarkService(), etc.) instead of using dependency injection or singletons. This wastes resources, prevents shared state, and makes testing difficult. Use a service locator (get_it) or Provider-based injection to share instances across the app.
HIGH	No Global Error Handling	Entire App	Errors are silently caught with catch (e) { debugPrint(...) } throughout the codebase. There is no global error boundary, no user-facing error messages in many places, and no crash reporting. Implement Flutter ErrorWidget, Zone error handling, and integrate Firebase Crashlytics for production error tracking.
HIGH	Massive Code Duplication	bookmark_service.dart, watchlist_service.dart	BookmarkService and WatchlistService share approximately 90% identical code (local/cloud storage, Firestore operations, merge logic). This violates DRY principles. Create a generic BaseCollectionService that handles CRUD, sync, and local fallback, then extend it for bookmarks and watchlist.

HIGH	No Reactive State for Data	All Screens	Provider is only used for AppConfig. Movie lists, bookmarks, and watchlist data are fetched independently on each screen with no shared reactive state. When a user bookmarks a movie on one screen, other screens do not update. Implement a proper state management solution (Riverpod, BLoC, or Provider with ChangeNotifier for data models).
HIGH	Dead Code: ApiService	api_service.dart	ApiService references URL constants from constants.dart, but that file is now empty ("API URLs removed - now using Firebase Firestore"). All API URL references (apiMovieTrendingUrl, apiSearchUrl, etc.) will resolve to null/empty, making the entire service non-functional. Either remove this dead code or document it as deprecated.
HIGH	Download Manager Platform-Specific	download_manager_service.dart	Hardcoded path /storage/emulated/0/Download/CM_Movies/ only works on Android. No iOS support using path_provider. No Android 10+ scoped storage permission handling (WRITE_EXTERNAL_STORAGE is deprecated). No check for storage availability or permission before download. Use path_provider getApplicationDocumentsDirectory() or getExternalStorageDirectory().
HIGH	No Admin Authorization on Backend	admin_panel_page.dart	Admin CRUD operations (add/edit/delete movies, genres, tags) are performed directly from the client with no backend validation. Any user who discovers the admin panel route can modify content. Firestore Security Rules should verify isAdmin custom claims before allowing write operations.

3. MEDIUM - Code Quality Issues

These issues affect code maintainability and can lead to bugs, performance problems, or inconsistent behavior. While not immediately dangerous, they should

be addressed to improve the overall health of the codebase and prevent futureregressions.

Severity	Issue	File	Description
MEDIUM	Duplicated DateTime Parsing	movie.dart, movie_detail.dart	_parseDateTime and _parseDateTimeDetail are identical functions in two different model files. Extract to a shared utility file (e.g., utils/date_parser.dart) to avoid maintenance burden and potential divergence.
MEDIUM	Inefficient Client-Side Search	firestore_content_service.dart	searchMovies() fetches up to 500 documents from Firestore and filters client-side. This approach is extremely expensive on Firestore reads (each read counts toward billing quota) and will not scale. Consider using Algolia, Typesense, or Firebase Extensions for full-text search. At minimum, implement debouncing and caching.
MEDIUM	Excessive Fallback Patterns	firestore_content_service.dart	Every Firestore query has try/catch with a fallback query without orderBy. This masks the real problem: missing composite indexes. Instead of working around missing indexes, create the required Firestore composite indexes using firestore.indexes.json and the Firebase Console. The fallback pattern doubles your Firestore read costs.
MEDIUM	No Pagination in Admin Panel	admin_panel_page.dart	getAllPosts(limit: 100) loads up to 100 items at once with no infinite scroll or load-more mechanism. As content grows, this will become slow and memory-intensive. Implement cursor-based pagination similar to the user-facing screens.

MEDIUM	Movie ID Inconsistency	movie.dart	Movie.fromMap uses docId parameter for Firestore documents, but toMap() includes the id field separately. When saving bookmarks locally via SharedPreferences, the id from toMap() is used. If docId and map["id"] differ, bookmark/watchlist lookups will fail. Ensure consistent ID handling throughout the data flow.
MEDIUM	No URL Validation	add_movie_page.dart, edit_movie_page.dart	Poster URL, Backdrop URL, and download link URL fields accept any string without validation. Invalid URLs will cause image loading failures and broken download links. Add URL format validation and optionally a URL preview feature.
MEDIUM	withOpacity Performance Issue	Multiple Files	Color(0xFFE50914).withOpacity(0.15) is called extensively in build methods. Each call creates a new Color object on every widget rebuild, causing unnecessary garbage collection. Pre-compute these colors as static constants or use Color.fromARGB() for better performance.
MEDIUM	SharedPreferences for Large Data	download_manager_service.dart, bookmark_service.dart	Storing serialized JSON lists (bookmarks, downloads) in SharedPreferences has size limits and performance issues. For download tasks especially, as the list grows, serialization/deserialization becomes slow. Consider using Hive, Isar, or sqflite for structured local storage.
MEDIUM	Missing Input Sanitization	add_movie_page.dart, admin_panel_page.dart	Admin forms accept raw text for genre names, tags, movie titles, etc. without sanitization. Special characters or extremely long strings could break Firestore operations or UI rendering. Add input length limits and character filtering.

MEDIUM	No Loading States for Background Ops	Multiple Screens	Some async operations (bookmark toggle, watchlist add) do not show loading indicators. Users may tap multiple times, causing duplicate operations. Add loading states and disable buttons during async operations.
---------------	---	------------------	--

4. LOW - Improvements and Best Practices

These are recommended improvements that follow Flutter and Dart best practices. While not causing immediate problems, implementing them will improve code quality, developer experience, and long-term maintainability of the project.

Severity	Issue	File	Description
LOW	No Test Files	Entire Project	The project has zero test files. No unit tests, widget tests, or integration tests exist. This makes refactoring risky and regressions likely. At minimum, add unit tests for services (BookmarkService, FirestoreContentService) and widget tests for key screens.
LOW	No CI/CD Pipeline	Project Root	No GitHub Actions or other CI/CD configuration. Automated testing, linting, and build verification are missing. Add a basic GitHub Actions workflow for flutter test, flutter analyze, and flutter build.
LOW	Hardcoded UI Strings	Multiple Screens	Many UI strings in admin screens ("Admin Panel", "Add Movie", "Delete Confirmation") are hardcoded in English instead of using the translation system. This creates inconsistency where some screens support bilingual (Myanmar/English) and others do not.

LOW	No Analytics/Crash Reporting	Entire App	No Firebase Analytics, Crashlytics, or any usage tracking. You have no visibility into how users interact with the app or what errors they encounter. Integrate Firebase Analytics for user behavior and Crashlytics for crash reporting.
LOW	Missing iOS Support	firebase_options.dart	DefaultFirebaseOptions throws UnsupportedError for iOS. The app cannot run on iOS devices. Add iOS Firebase configuration if cross-platform support is desired.
LOW	Theme Color Hardcoded	Multiple Files	Color(0xFFE50914) (Netflix red) is repeated dozens of times across files instead of referencing the theme. If the brand color changes, every file needs updating. Define it once in the theme or as a constant.
LOW	No README.md	Project Root	No project documentation exists. A README should include: setup instructions, Firebase configuration steps, environment variables needed, and build/run commands. This is especially important for a project with Firebase dependencies.
LOW	firestore.indexes.json May Be Incomplete	firestore.indexes.json	The file exists but likely does not cover all the composite indexes needed by the queries with fallback patterns. Every fallback query indicates a missing index. Review all Firestore queries and add the required composite indexes.
LOW	No Deep Linking	Entire App	No support for sharing movie links or opening the app from a URL. This is a common feature for movie apps and would improve user engagement. Consider implementing Firebase Dynamic Links or universal links.

LOW	Edit Page Always Clears Backdrop	edit_movie_page.dart:121	When editing a movie, backdrop is always set to null (line 121: "backdrop": null). This means every edit operation removes the backdrop image. The backdrop URL should be loaded from the existing data and preserved unless explicitly cleared by the user.
-----	---	--------------------------	--

5. Project Structure Overview

The project follows a reasonable Flutter project structure with separation of concerns between models, services, and UI screens. However, there are some organizational improvements that could enhance maintainability and scalability as the codebase grows.

Directory	Purpose	Notes
lib/main.dart	App entry point, theme configuration	Theme definitions are very long (300 lines). Consider extracting to a separate theme file.
lib/app/core/models/	Data models (Movie, MovieDetail, TagAndGenres, MovieYear)	Models are well-structured. MovieYear may be unused after Firestore migration.
lib/app/core/services/	Business logic services (7 services)	High duplication between BookmarkService and WatchlistService. ApiService is dead code.
lib/app/ui/screens/	Page-level widgets (16 screens)	Some screens are very large (edit_movie_page.dart = 787 lines). Consider breaking into smaller widgets.
lib/app/ui/home/	Home tab components	Good separation of home screen from tab navigation.
lib/app/ui/components/	Reusable UI components	Only 3 components. Many reusable patterns (section titles, multi-select chips) are duplicated across screens.

lib/more_libs/setting/	App configuration and state	AppConfig handles too many responsibilities: auth, theme, language, translations. Should be split.
android/	Android native configuration	google-services.json should not be in version control. Use environment-based configuration.

6. Priority Recommendations

Based on the findings of this code review, here is a prioritized action plan organized by urgency. The items are ordered from most critical to least critical, reflecting the order in which they should be addressed to maximize security and stability improvements in the shortest time.

Phase 1: Immediate Action (This Week)

1. **Revoke the exposed GitHub token** immediately and generate a new one. Never share tokens in chat or commit them to repositories.
2. **Add a .gitignore file** to the project root. Use the Flutter template and add google-services.json and firebase_options.dart to the ignore list. Remove these files from git history using git filter-branch or BFG Repo Cleaner.
3. **Configure Firestore Security Rules** that validate authentication and admin roles. At minimum: users can only read/write their own bookmarks; only admin users can modify movies/genres/tags collections.
4. **Remove the hardcoded admin email** from app_config.dart. Move admin detection to Firestore Custom Claims or a secure admin lookup collection that is not visible in client code.
5. **Fix the edit_movie_page.dart backdrop bug** that always sets backdrop to null on save. This is causing data loss for every edit operation.

Phase 2: Short Term (Next 2 Weeks)

1. **Refactor BookmarkService and WatchlistService** into a shared generic base class. This eliminates approximately 200 lines of duplicated code and makes

adding new collection types trivial.

2. **Implement proper dependency injection** using `get_it` or `Provider`. Create service singletons that are shared across screens instead of creating new instances on every navigation.
3. **Create all required Firestore composite indexes** and remove the fallback query patterns. This will halve your Firestore read costs and eliminate silent failures.
4. **Remove dead ApiService code** and the empty `constants.dart` file. Dead code confuses new developers and increases maintenance burden.
5. **Fix the Download Manager** to use `path_provider` for cross-platform storage paths and handle Android scoped storage permissions properly.

Phase 3: Medium Term (Next Month)

1. **Implement a proper state management solution** (Riverpod recommended) for reactive data. All screens should share the same data state so that changes propagate automatically.
2. **Add Firebase Analytics and Crashlytics** for production monitoring. You need visibility into user behavior and crash reports to maintain a healthy application.
3. **Replace SharedPreferences-based storage** for bookmarks and downloads with Hive or Isar. `SharedPreferences` is not designed for large structured data and will degrade as the dataset grows.
4. **Add unit and widget tests** for critical services and screens. Start with `BookmarkService`, `FirestoreContentService`, and the login flow. Target at least 50% code coverage.
5. **Set up CI/CD with GitHub Actions** for automated flutter test, flutter analyze, and build verification on every pull request.

Phase 4: Long Term (Ongoing)

1. **Implement full-text search** using Algolia or Firebase Extensions instead of client-side filtering. This is critical for scalability as your content library grows beyond a few hundred items.

2. **Split AppConfig into focused providers:** AuthProvider, ThemeProvider, LanguageProvider, and TranslationProvider. The current 595-line file is doing too much.
3. **Extract shared UI components** (section titles, multi-select chips, form fields) into the components/ directory to reduce duplication across screens.
4. **Add deep linking support** for sharing movie links. This improves discoverability and user engagement significantly.
5. **Implement offline-first architecture** with local caching (Hive/Isar) and background sync. Users in Myanmar may experience intermittent connectivity, and the current app requires internet for all content.

7. What Is Working Well

Despite the issues identified above, the project has several strengths that are worth acknowledging. These positive aspects provide a solid foundation for the improvements recommended in this review, and they demonstrate good development instincts that should be maintained as the codebase evolves.

Netflix-inspired UI Design: The app has a polished, consistent dark theme with the Netflix-style red accent color. The Material Design 3 integration with custom color schemes for both light and dark modes shows attention to visual quality and user experience.

Firebase Integration: The migration from REST API to Firestore is a smart architectural decision. Firestore provides real-time updates, offline support, and eliminates the need for a separate backend server. The cursor-based pagination implementation is well done.

Local/Cloud Data Sync: The bookmark and watchlist services implement a thoughtful local-first approach with cloud sync on login. This is a good UX pattern for users who may be offline or not logged in. The merge logic prevents data loss during sync.

Bilingual Support: The Myanmar/English translation system is comprehensive with 90+ translation keys covering all major features. This shows consideration for the local user base.

Download Manager Architecture: The download manager with pause/resume/retry functionality and progress tracking is well-designed. The state machine approach (idle/downloading/paused/completed/failed) is clean and maintainable.

Age Rating Gate: The 18+ content filtering with a confirmation gate is a responsible feature that shows awareness of content appropriateness, especially important for a movie app accessible to all ages.

Admin Panel: The comprehensive admin panel with movie/series CRUD, genre/tag/collection management, and search functionality gives content managers full control without needing direct Firestore access.

Search with Filters: The multi-dimensional search (keyword, genre, type, year, rating, sort) provides powerful content discovery. The filter UI with active filter badges is intuitive.